

CI/CD Assignment Documentation

Laravel GitHub Actions Pipeline

Date: September 19, 2026

Project Overview

This project implements a complete CI/CD pipeline for a Laravel application using GitHub Actions. The pipeline includes automated testing, staging deployment, and production deployment stages.

Phase 1: Task CRUD Implementation

1.1 Database Migration

File: database/migrations/2026_09_19_024220_create_tasks_table.php

Schema includes: id, title, description (nullable), status (default: pending), timestamps

1.2 Task Model

File: app/Models/Task.php

Fillable fields: title, description, status

1.3 Task Controller

File: app/Http/Controllers/TaskController.php

Methods: index, create, store, show, edit, update, destroy

1.4 Routes

Route::resource('tasks', TaskController::class);

1.5 Blade Views

- resources/views/tasks/index.blade.php (Task list)
- resources/views/tasks/create.blade.php (Create form)
- resources/views/tasks/edit.blade.php (Edit form)
- resources/views/tasks/show.blade.php (Task details)

1.6 Feature Tests

File: tests/Feature/TaskTest.php

Tests included:

- test_can_list_tasks()
- test_can_create_task()
- test_can_update_task()
- test_can_delete_task()

Testing: SQLite in-memory database (phpunit.xml)

Phase 2: Deployment Script

2.1 deploy.sh

File: deploy.sh (7 sequential steps)

Step 1: Turn on maintenance mode (php artisan down)

Step 2: Pull latest changes (git pull origin main)

Step 3: Install dependencies (composer install --no-interaction --prefer-dist --optimize-autoloader --no-dev)

Step 4: Run database migrations (php artisan migrate --force)

Step 5: Clear and cache routes/config (php artisan optimize)

Step 6: Restart queues (php artisan queue:restart)

Step 7: Turn off maintenance mode (php artisan up)

Phase 3: GitHub Actions Pipeline

3.1 Workflow Configuration

File: .github/workflows/deploy.yml

3.2 Pipeline Stages

Stage 1: Build

- Checkout code
- Setup PHP 8.3
- Install Composer dependencies

- Upload vendor directory as artifact

Stage 2: Test (needs: build)

- Download vendor artifact
- Setup PHP environment
- Copy .env.example to .env
- Generate application key
- Create SQLite database
- Run PHPUnit tests

Stage 3: Staging (needs: test)

- Runs after successful tests
- Simulates staging deployment

Stage 4: Production (needs: staging)

- Conditional: only on main branch
- Uses production environment
- Echoes 7-step deployment process

Phase 4: Testing & Simulation

4.1 Red Pipeline (Failure)

Steps to create failing test:

1. Open tests/Feature/TaskTest.php
2. Go to line 21 in test_can_list_tasks()
3. Change: `$response->assertStatus(200);`
4. To: `$response->assertStatus(404);`
5. Commit & push to GitHub
6. Observe pipeline failure
7. Take screenshot of red pipeline

4.2 Green Pipeline (Success)

Steps to fix and pass tests:

1. Revert change in tests/Feature/TaskTest.php
2. Change back: `$response->assertStatus(200);`
3. Commit & push to GitHub
4. Observe successful pipeline
5. Take screenshot of green pipeline

Key Features & Highlights

- ' Complete CRUD implementation for Task entity
- ' Comprehensive test coverage with 4 feature tests
- ' SQLite in-memory testing configuration
- ' Four-stage sequential CI/CD pipeline
- ' Artifact caching for vendor dependencies
- ' Conditional production deployment (main branch only)
- ' Environment-specific deployment strategies
- ' Automated testing before deployment

Conclusion

This implementation demonstrates a production-ready CI/CD pipeline for Laravel applications. The pipeline ensures code quality through automated testing, provides staging environment validation, and implements safe production deployment practices with conditional execution and environment protection.